

Documentacao Detalhada

Aula 9 - Flask Requests

Material de apoio do projeto: app.py, dados.json, CSS, JavaScript e templates Jinja2.

Janaína Duarte

1. Visao geral do projeto

Este projeto da Aula 9 demonstra duas maneiras diferentes de o navegador enviar informacoes ao servidor Flask e receber respostas. O objetivo pedagogico e consolidar conceitos vistos nas aulas anteriores (templates HTML na Aula 8, formularios na Aula 6) e introduzir JSON, jsonify, request.form, request.get_json e fetch no JavaScript.

O aluno percorre dois fluxos independentes. No Fluxo 1, a validacao da senha ocorre no servidor quando o formulario HTML e enviado por POST; o Flask le os dados com request.form. No Fluxo 2, a pagina de senha nao recarrega para validar: o JavaScript envia um fetch para uma rota de API que responde com jsonify; se a senha estiver correta, o navegador e redirecionado para uma pagina com o link oficial do site do Cotemig (<https://cotemig.com.br/>).

Estrutura de pastas do projeto:

```
Aula9-Requests/
  app.py                (servidor Flask - rotas e logica)
  dados.json            (configuracao e textos)
  gerar_documentacao_pdf.py
  static/
    css/style.css       (estilos)
    js/main.js          (fetch no fluxo 2)
  templates/
    base.html           (layout pai - heranca Jinja)
    index.html          (pagina inicial)
    fluxo1_inicio.html
    fluxo1_senha.html
    fluxo2_inicio.html
    fluxo2_senha.html
    cotemig.html         (link externo Cotemig)
```

Senhas de demonstracao para a aula:

- Fluxo 1: aluno2026
- Fluxo 2: cotemig

Como executar: python app.py e abrir <http://127.0.0.1:5000/>

2. Arquivo app.py

O app.py é o coração do sistema. Ele cria a aplicação Flask, define todas as rotas (URLs), carrega o arquivo dados.json, valida senhas e decide se a resposta será uma página HTML (render_template) ou dados JSON (jsonify).

2.1 Importações e configuração inicial

```
import json
from pathlib import Path
from flask import Flask, jsonify, render_template, request

app = Flask(__name__)
ARQUIVO_DADOS = Path(__file__).parent / "dados.json"
```

Flask: framework web. jsonify: converte dicionário Python em resposta JSON. render_template: monta HTML a partir de arquivos na pasta templates. request: objeto que representa a requisição HTTP atual (formulário, JSON, método). Path(__file__).parent garante que dados.json seja encontrado na mesma pasta do app.py, mesmo se o comando for executado de outro diretório.

2.2 Função carregar_dados()

Abre dados.json com encoding utf-8 e retorna um dicionário Python. Esse dicionário é usado nas rotas para comparar senhas e passar textos aos templates. Separar dados em JSON permite alterar mensagens e senhas sem mexer na lógica Python - útil em aula.

2.3 Função dados_publicos()

Retorna apenas site_cotemig e mensagens, sem o objeto senhas. A rota /api/info usa essa função para ensinar que APIs públicas não devem vaziar credenciais. É um ponto importante de segurança para comentar com a turma.

2.4 Rota index - página inicial (/)

Método GET implícito. Renderiza index.html passando dados=carregar_dados(). A página inicial apresenta os dois fluxos e um link para /api/info.

2.5 Fluxo 1 - rotas /fluxo1/ e /fluxo1/senha

fluxo1_inicio: apenas exibe HTML explicativo com link para a página de senha.

fluxo1_senha aceita GET e POST. No GET, mostra o formulário vazio. No POST, request.form.get('senha') lê o campo enviado pelo formulário. Compara com dados['senhas']['fluxo1']. Se igual, define variável sucesso; senão, erro. O mesmo template é renderizado de novo, mas agora com mensagem de alerta - padrão Post/Redirect/Get simplificado sem redirect, adequado para iniciantes.

2.6 Fluxo 2 - rotas /fluxo2/

fluxo2_inicio e fluxo2_senha: páginas HTML. fluxo2_cotemig: página final com link para

<https://cotemig.com.br/>. O acesso a essa pagina no fluxo ideal ocorre apos validacao via API; a rota em si nao exige senha novamente (em producao usaria-se sessao ou token).

2.7 API /api/info

Retorna jsonify(dados_publicos(...)). O navegador recebe JSON puro. Serve para mostrar jsonify isolado e para o botao Ver /api/info na pagina inicial.

2.8 API /api/validar-senha (POST)

request.get_json(silent=True) le o corpo JSON enviado pelo fetch. Espera fluxo: fluxo2 e senha. Se corretos, jsonify com ok: true e redirect: /fluxo2/cotemig. Se incorretos, jsonify com ok: false e codigo HTTP 401 (nao autorizado). O codigo 401 ensina que APIs comunicam estado por status HTTP, nao so pelo JSON.

2.9 app.run(debug=True)

Inicia servidor local na porta 5000. debug=True recarrega ao salvar arquivos e mostra rastreamento de erros - apenas em desenvolvimento, nunca em producao.

3. Arquivo dados.json

Arquivo de configuracao em formato JSON (JavaScript Object Notation). JSON usa chaves entre aspas duplas, valores string, numero, booleano, array ou objeto. Qualquer linguagem le JSON; por isso e padrao em APIs.

3.1 Objeto senhas

- fluxo1: aluno2026 - senha do formulario POST
- fluxo2: cotemig - senha validada pela API

3.2 site_cotemig

URL <https://cotemig.com.br/> usada no template cotemig.html. O projeto nao incorpora o site dentro da aplicacao; apenas oferece um link externo (target=_blank).

3.3 Objeto mensagens

Centraliza textos exibidos nas telas. Vantagens: um unico lugar para editar frases; templates mais limpos com {{ dados.mensagens.fluxo1_titulo }}; prepara alunos para internacionalizacao (i18n) no futuro.

4. Arquivo static/css/style.css

Folha de estilos simples e pedagogica. Flask serve arquivos da pasta static/ na URL /static/... O base.html referencia com `url_for('static', filename='css/style.css')`.

4.1 Reset e body

Seletor `*` zera margin e padding e usa box-sizing border-box para larguras previsiveis. body define fonte Segoe UI, fundo cinza-azulado (#f0f4f8), cor do texto e altura minima 100vh.

4.2 Layout .container

Centraliza conteudo com max-width 520px - layout de cartao estreito, bom para formularios.

4.3 Componentes principais

- .card: caixa branca com borda, sombra leve e padding
- .btn e .btn-secondary: botoes azul primario e cinza secundario
- .alert-erro, .alert-ok, .alert-info: feedback visual
- .nav-links: flexbox para alinhar links de navegacao
- .hidden: display none - usado no JS para esconder mensagem de erro
- .link-externo: destaque do link do Cotemig

Nao ha responsividade avancada nem framework CSS; proposital para a turma focar em Flask.

5. Arquivo static/js/main.js

JavaScript carregado em todas as paginas pelo base.html, mas so atua no Fluxo 2 (quando existe #form-senha-fluxo2). Se o elemento nao existir, a funcao retorna cedo.

5.1 DOMContentLoaded

Garante que o HTML ja foi montado antes de buscar elementos. Evita erro de null.

5.2 Evento submit do formulario

event.preventDefault() impede o envio tradicional do formulario (que recarregaria a pagina). A validacao passa a ser assincrona via fetch.

5.3 fetch para /api/validar-senha

```
fetch("/api/validar-senha", {
  method: "POST",
  headers: { "Content-Type": "application/json" },
  body: JSON.stringify({ fluxo: "fluxo2", senha: senha })
})
```

fetch e a API moderna do navegador para HTTP (equivalente pedagogico a biblioteca requests do Python, mas no cliente). Content-Type application/json informa ao Flask que o corpo e JSON (request.get_json). resposta.json() converte a resposta em objeto.

5.4 Tratamento de sucesso e erro

Se dados.ok for true, window.location.href redireciona para /fluxo2/cotemig. Senao, exibe mensagem em #msg-erro removendo classe hidden. try/catch cobre falha de rede. finally restaura o botao - boa pratica de UX.

6. Templates Jinja2 e heranca

Jinja2 e o motor de templates do Flask. Heranca significa: um arquivo base define a estrutura comum; os filhos estendem com `{% extends %}` e preenchem `{% block %}`.

6.1 base.html - template pai

Define esqueleto HTML5: head com charset UTF-8, viewport, titulo dinamico no block titulo, link do CSS, header com block cabecalho e subtítulo, main com block conteudo, footer com block rodape, script main.js e block scripts_extra para scripts adicionais por pagina.

`url_for('static', filename='...')` gera URL correta do arquivo estatico. `url_for('nome_da_funcao')` gera URL da rota pelo nome da funcao Python em app.py.

6.2 index.html

Pagina menu com tres cards: apresentacao Fluxo 1, Fluxo 2 e link para API. Usa `url_for('fluxo1_inicio')`, `url_for('fluxo2_inicio')`, `url_for('api_info')`.

6.3 fluxo1_inicio.html

Estende base. Cabecalho vem de `dados.mensagens.fluxo1_titulo` (passado pelo Flask). Explica POST e request.form. Botao leva a `fluxo1_senha`.

6.4 fluxo1_senha.html

`{% if erro %}` e `{% if sucesso %}` mostram alertas condicionais. Form method POST action `url_for('fluxo1_senha')` - envia para a mesma rota que processa. Campo `name=senha` deve coincidir com `request.form.get('senha')`.

6.5 fluxo2_inicio.html e fluxo2_senha.html

Inicio analogo ao fluxo 1. Senha: form id `form-senha-fluxo2` sem method/action porque o JS intercepta. div `msg-erro` com class `hidden` inicialmente.

6.6 cotemig.html

Pagina de sucesso do Fluxo 2. Link `href={{ dados.site_cotemig }}` `target=_blank` abre site oficial em nova aba. `rel=noopener noreferrer` e boa pratica de seguranca.

7. Fluxo de dados passo a passo

7.1 Fluxo 1 - request.form

- Usuario abre /fluxo1/
- Clica Ir para a senha -> GET /fluxo1/senha
- Digita senha e clica Validar -> POST /fluxo1/senha
- Flask le request.form, compara com dados.json
- Re-renderiza fluxo1_senha.html com erro ou sucesso

7.2 Fluxo 2 - fetch + jsonify

- Usuario abre /fluxo2/ e depois /fluxo2/senha
- Digita senha; JS envia POST JSON para /api/validar-senha
- Flask responde jsonify; JS le dados.ok
- Redirect para /fluxo2/cotemig com link <https://cotemig.com.br/>

8. Tabela comparativa para a aula

Aspecto	Fluxo 1	Fluxo 2
Envio de dados	Formulario HTML POST	fetch + JSON
Leitura Flask	request.form	request.get_json
Resposta	HTML (template)	JSON (jsonify)
Pagina recarrega	Sim	Nao (ate redirect)
Pagina final	Mensagem na mesma	cotemig.html + link

9. Sugestoes de condução em sala

- Demonstrar /api/info no navegador antes dos fluxos
- Abrir Ferramentas do Desenvolvedor (Rede) no Fluxo 2 para ver o POST JSON
- Comparar com Aula 8: render_template com painel completo
- Discutir por que senhas em JSON de aula nao sao seguras em producao
- Mencionar biblioteca requests do Python como proximo passo (cliente servidor)

10. Referencias

Documentacao Flask: <https://flask.palletsprojects.com/>

Site Cotemig: <https://cotemig.com.br/>

MDN fetch API: https://developer.mozilla.org/pt-BR/docs/Web/API/Fetch_API